HND IN COMPUTER SCIENCE

Module: Web Application Development

Full-Stack Web Application Development Project

AI-Powered Study Planner and Learning Path Optimization System

Student Name	Thanushan Santhathevan
Student ID	UB036HNDCS
Programme	HND in Computer Science
Module	Web Application Development
Assessment Type	Individual Project
Final Evaluation	Sunday, 6 December 2026
GitHub Repository	https://github.com/ThanusanS/study-planner
Live Application	https://studyplanner.thanusan.dev

Table of Contents

1. Introduction 3

2. Project Objectives and Scope 3

3. System Analysis and Requirements 4

4. UI/UX Design and Front-End Development 5

5. Software Architecture and Technology Selection 9

6. Back-End Development and API Integration 10

7. Database Design and Integration 13

8. Authentication, Security, and Payment or API Integration 14

9. Testing and Quality Assurance 16

10. Deployment and Production Readiness 17

11. GitHub Repository and Development Process 18

12. Conclusion 18

13. References 18

14. Appendices 20

1. Introduction

Many students struggle to organise coursework, revise consistently and judge how much effort each subject needs. Generic planners store tasks but do not help learners understand material, rehearse for examinations or plan a learning path. This project, the **AI-Powered Study Planner and Learning Path Optimization System**, addresses that gap with a single web application that combines traditional planning (subjects, tasks, exams and focus timers) with generative-AI tools that produce quizzes, notes, learning roadmaps and tutoring explanations.

The application is a full-stack, cloud-hosted system built with React 18 and TypeScript on the front end and Appwrite as the back-end platform (authentication, database, real-time messaging and serverless functions). The Google Gemini 2.5 Flash model supplies the AI features, and Stripe in test mode handles subscription payments. The source code is held at https://github.com/ThanusanS/study-planner and the working system is available at https://studyplanner.thanusan.dev. This report documents the analysis, design, implementation, testing and deployment of the system in line with the assessment brief.

2. Project Objectives and Scope

2.1 Objectives

- Provide one organised workspace for subjects, topics, tasks and exam countdowns.
- Help learners build focus habits through Pomodoro sessions, streaks and progress analytics.
- Generate personalised AI quizzes, short or full notes, roadmaps and tutor explanations on demand.
- Support collaborative accountability through live co-study rooms with shared timers and chat.
- Control AI usage fairly through a subscription and credit system with an auditable ledger.
- Meet professional standards for security, testing, deployment and documentation.

2.2 Scope

In scope are the web application, its Appwrite back end, four serverless functions, the Gemini integration and Stripe test-mode payments. The system is delivered as a responsive installable web application. Out of scope are a native mobile application, offline synchronisation, calendar integration and real-money payment processing, which are recorded as future enhancements.

3. System Analysis and Requirements

3.1 System description and problem statement

The system is a multi-user study management platform. Each learner registers, creates subjects with colour coding, adds topics, schedules tasks and exams, studies with a Pomodoro timer and receives dashboard analytics. AI modules reduce the time spent preparing study material. The core problem is fragmentation: students use separate tools for planning, timing, note-making and revision, which hides progress and wastes time.

3.2 Target users and roles

The primary users are school, college and university students. Access is governed by three plan-based roles: **Free** (core planning tools), **Scholar Pro** (adds AI study planner, notes and AI Tutor) and **Elite Premium** (adds Tutor Plus, personalised roadmaps and a larger monthly credit allowance). A Stripe webhook updates the plan role after payment.

3.3 Functional requirements

ID	Requirement
FR1	Register, log in and log out using email/password or Google sign-in.
FR2	Create, read, update and delete subjects, topics, tasks and exams.
FR3	Toggle task completion; filter, group, reschedule, archive and bulk-delete tasks and exams.
FR4	Display exam countdowns and highlight exams due within three days.
FR5	Run Pomodoro sessions, store history by subject and show daily, weekly and monthly statistics.
FR6	Show dashboard progress, streaks and study-time breakdown.
FR7	Generate, save, retake, edit and delete AI quizzes; evaluate written answers.
FR8	Generate short and full notes, quick and detailed roadmaps, and tutor explanations.
FR9	Analyse uploaded PDF, image or text study material (AI Doc Hub).
FR10	Create and join public or private co-study rooms with synchronised timer and chat.
FR11	Subscribe to plans, deduct credits per AI action, show ledger and downloadable receipts.
FR12	Send browser notifications for task reminders, exam alerts and completed sessions.

3.4 Non-functional requirements

- **Security:** per-user data isolation, hashed passwords, validated input and no secrets in source code.
- **Performance:** paginated, indexed queries; first contentful paint under 1.5 seconds as targeted in the repository documentation.
- **Usability and responsiveness:** consistent layout, light and dark themes and mobile-first design.
- **Scalability and availability:** managed cloud back end and static front-end hosting designed for large monthly user volumes.

- **Maintainability:** modular TypeScript components, a service layer and documented setup guides.

3.5 Use case model

Figure 1 shows the use cases. The student interacts with every module, the Gemini API supports the AI use cases and Stripe supports billing.

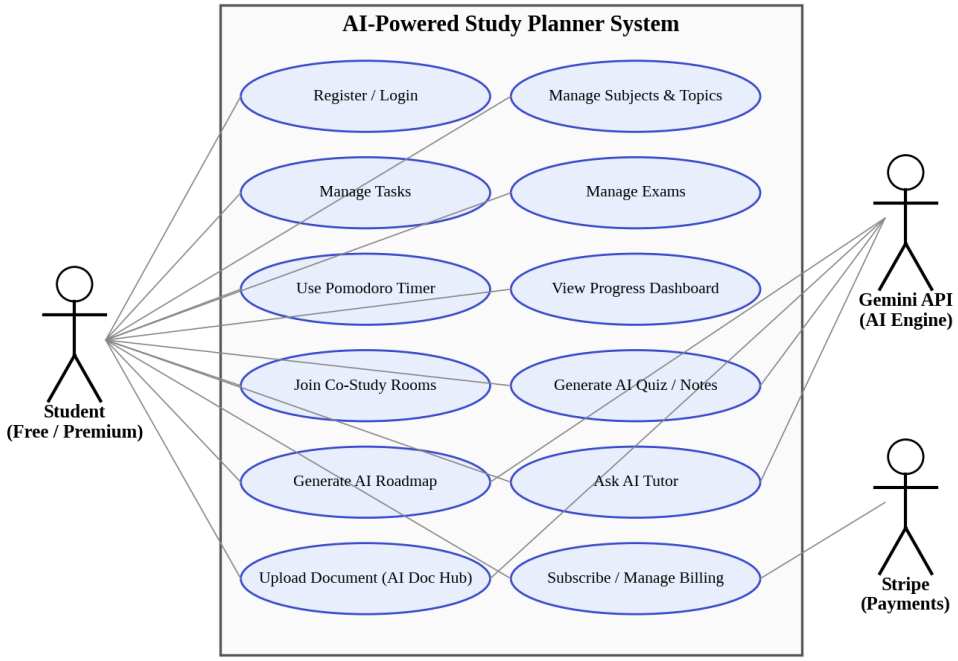


Figure 1: Use case diagram

4. UI/UX Design and Front-End Development

The interface follows a dashboard pattern: a fixed left navigation lists Dashboard, Subjects, Tasks, Exams, Pomodoro, Co-Study Rooms, AI Doc Hub, AI Quiz, AI Notes, AI Roadmap, AI Tutor and Billing & Plans, with the user profile and logout control at the bottom. Content is placed in rounded cards with a consistent indigo and violet palette, clear typography and gradient hero panels. Each module repeats the same structure (header card, statistic chips, tools and history), which keeps the experience predictable. A dark theme is available through the theme toggle.

The front end is a React 18 and TypeScript single-page application built with Vite. Tailwind CSS utility classes provide responsive layouts, while shadcn/ui supplies accessible components. Forms use required-field markers, controlled inputs and inline validation. Sonner displays toast feedback and lucide-react provides icons. Tasks and exams can be viewed as a list, table or calendar, and updates appear instantly through optimistic state changes. The browser offers an Install button, confirming that the application is delivered as

an installable web application.

The public landing page (Figure 2) introduces the product, and the login and registration page (Figure 3) is the entry point. The main user journey is: register, create subjects and topics, add tasks and exams, study with Pomodoro, review analytics, then generate AI study material.

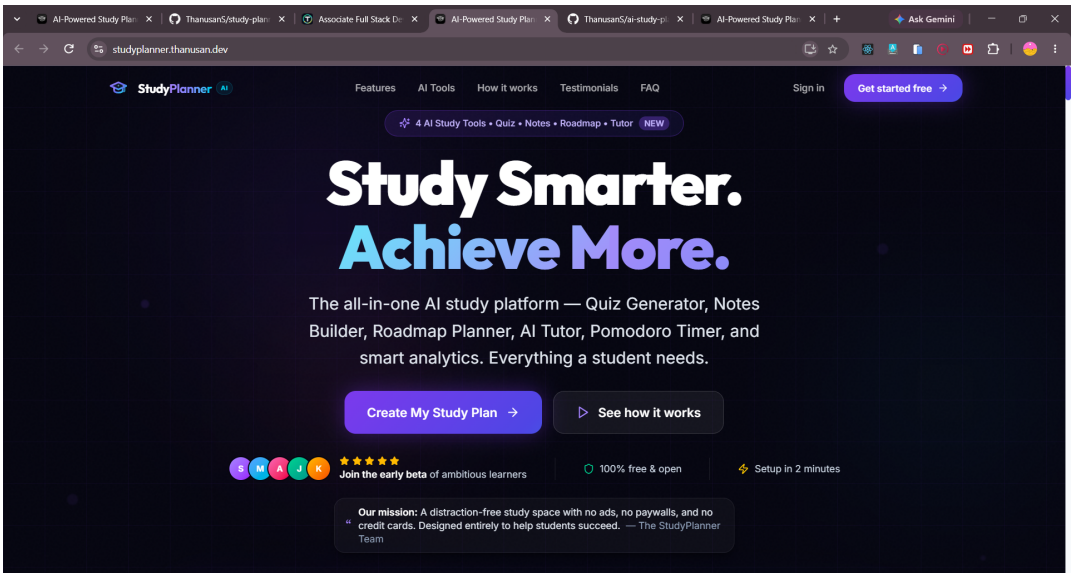


Figure 2: Landing page of the deployed application

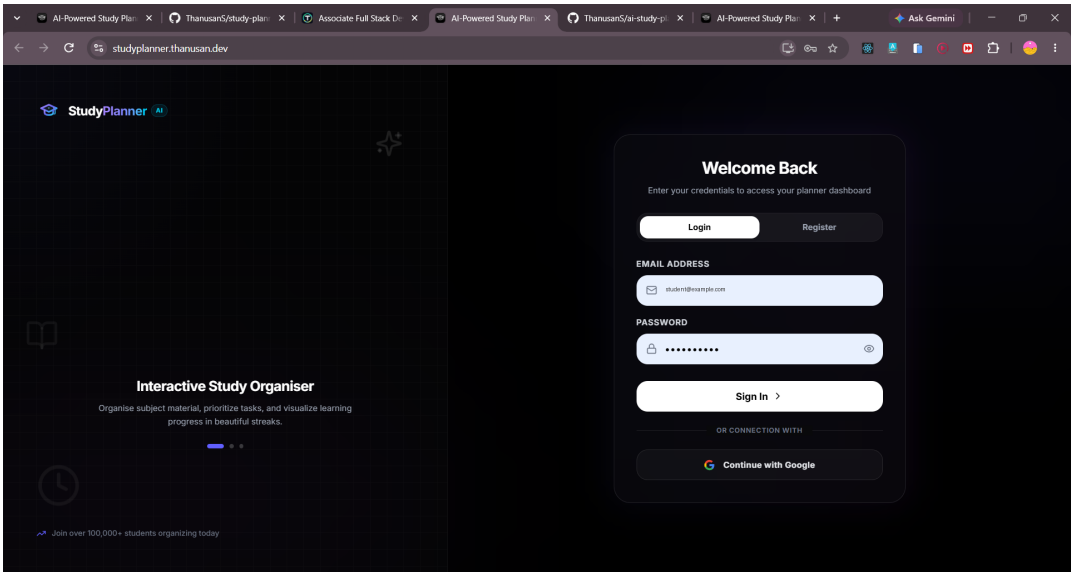


Figure 3: Login and registration page (email redacted)

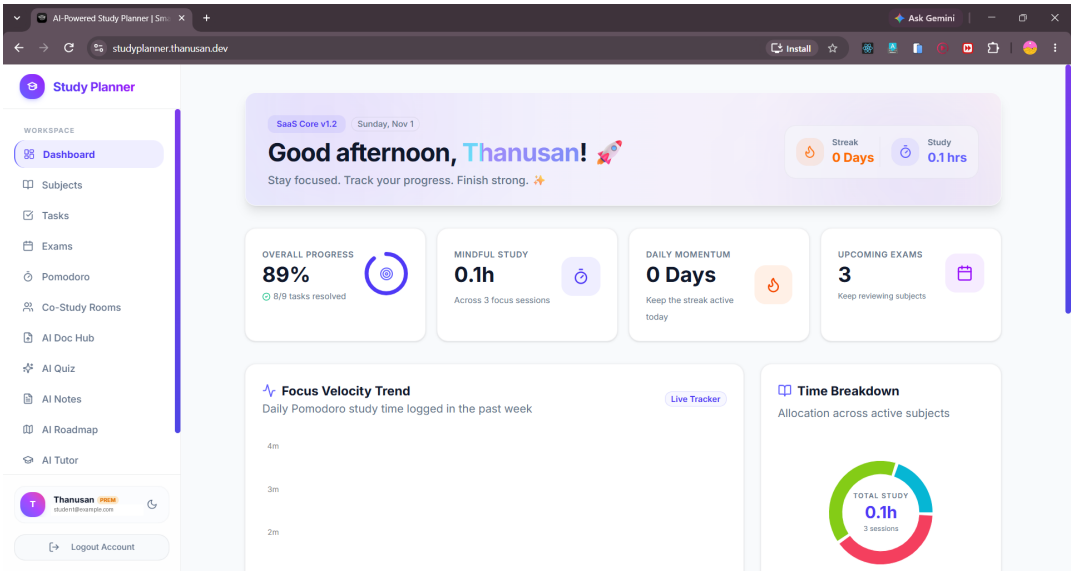


Figure 4: Dashboard with progress, study time and streak indicators

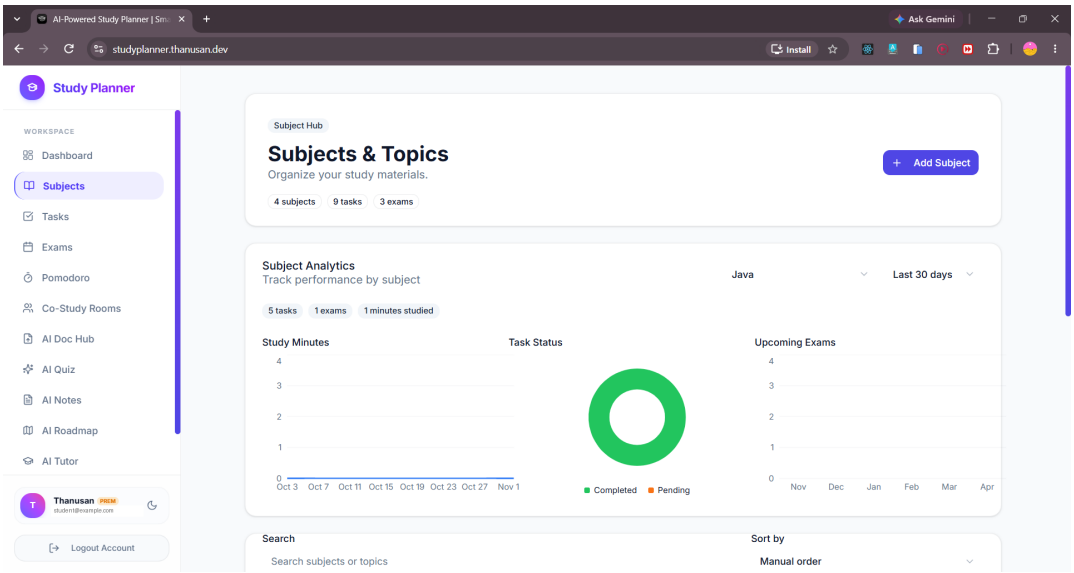


Figure 5: Subjects and Topics hub with analytics

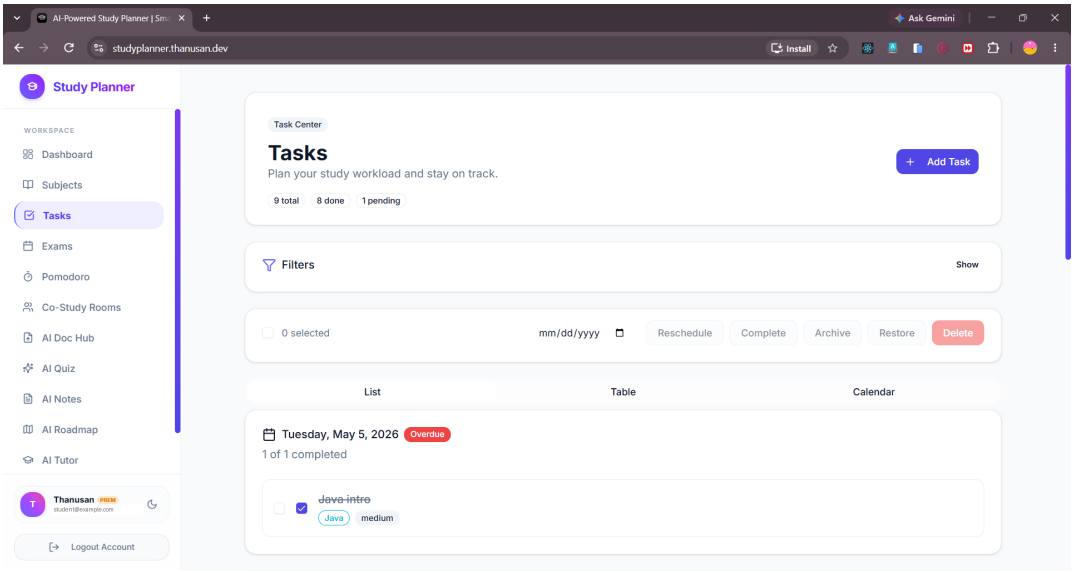


Figure 6: Task Center with filters, bulk actions and list, table and calendar views

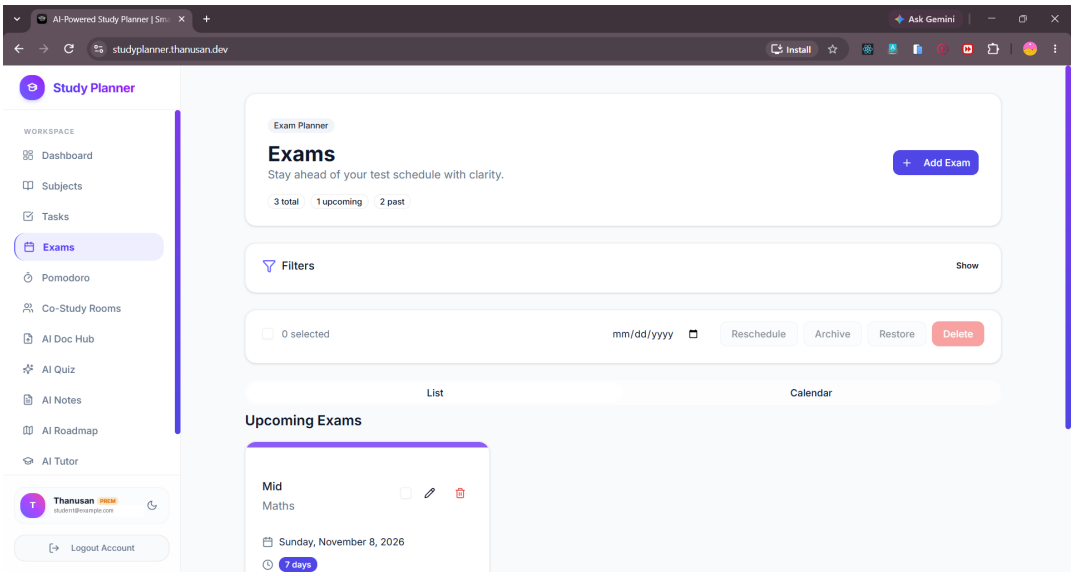


Figure 7: Exam planner with countdown badge

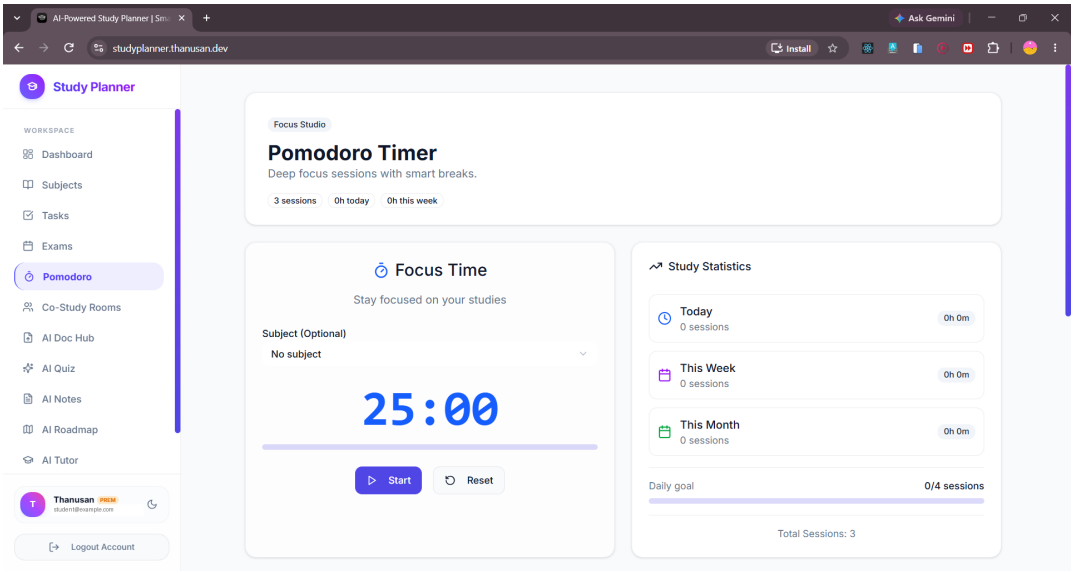


Figure 8: Pomodoro timer with study statistics

5. Software Architecture and Technology Selection

The system uses a **three-tier client-server architecture** supported by serverless functions and third-party services (Figure 9). The presentation tier is the React application hosted on Vercel. The application tier is Appwrite Cloud, which provides authentication, the database API, real-time subscriptions and function execution. The data tier is the Appwrite database with collections, indexes and document-level permissions.

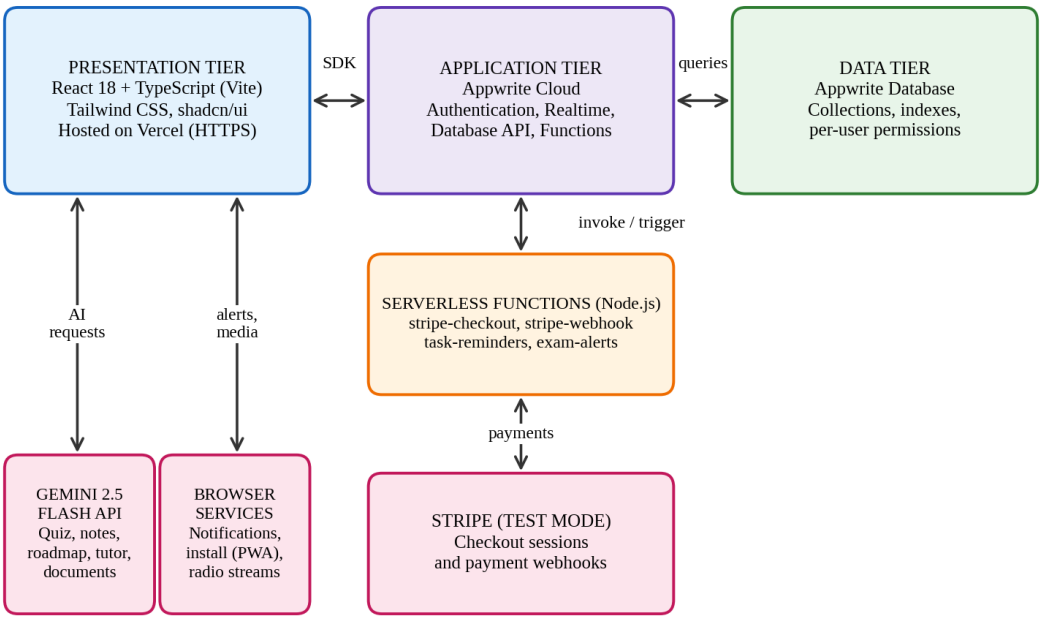


Figure 9: Software architecture diagram (three tiers, serverless functions and external services)

5.1 Component responsibilities and communication

The browser communicates with Appwrite over HTTPS through the Appwrite SDK from a service layer, so components never contain query logic. Real-time subscriptions deliver chat messages and presence events in study rooms. Serverless functions run on the server side: *stripe-checkout* creates checkout sessions, *stripe-webhook* confirms payments and updates plans, and *task-reminders* and *exam-alerts* generate reminder and alert notifications. AI requests are sent to the Gemini 2.5 Flash API, and each successful generation is recorded in the credit ledger.

5.2 Technology stack and justification

Layer	Technology	Justification
Front end	React 18, TypeScript, Vite, Tailwind CSS, shadcn/ui	Component reuse, type safety, fast builds and responsive styling.
Back end	Appwrite (Auth, Databases, Realtime, Functions)	Secure managed services shorten development and scale with demand.
AI	Google Gemini 2.5 Flash	Low latency, low cost, supports text and document or image input.
Payments	Stripe (test mode)	Industry-standard checkout and webhooks with a safe sandbox.
Hosting	Vercel, Git and GitHub, pnpm	Continuous deployment from the repository and HTTPS by default.

Maintainability comes from modular components and a service layer; scalability from managed infrastructure, pagination and indexes; and security from server-side permissions and keeping payment secrets inside functions.

6. Back-End Development and API Integration

The back end is organised as Appwrite collections accessed through service modules (tasks, subjects, exams, Pomodoro, quiz history, billing and rooms) plus four Node.js serverless functions. Each service exposes create, list, update and delete operations, applies server-enforced permissions and returns success or error results that the interface turns into toast messages.

Business logic includes grouping tasks by due date, computing completion percentage, streaks and weekly totals, flagging exams within three days, rescheduling in bulk and enforcing plan limits. Before an AI action runs, the application checks the credit balance; after success it deducts credits (short notes 2, full notes 5, tutor explain 3, roadmap 4, quiz 2) and writes a ledger entry with transaction ID, description, amount and timestamp. A monthly reset restores the plan allowance, for example 3000 credits for Elite Premium, with the next refill shown on the billing page.

Error handling covers failed network calls, empty required fields, insufficient credits and API failures; the user receives a clear message and no credits are deducted for a failed generation. The AI Doc Hub accepts PDF, PNG, JPG or TXT files, and the output is shown in the Study Studio panel and saved to history. Saved roadmaps and notes can be viewed, edited, downloaded or deleted (Figure 15).

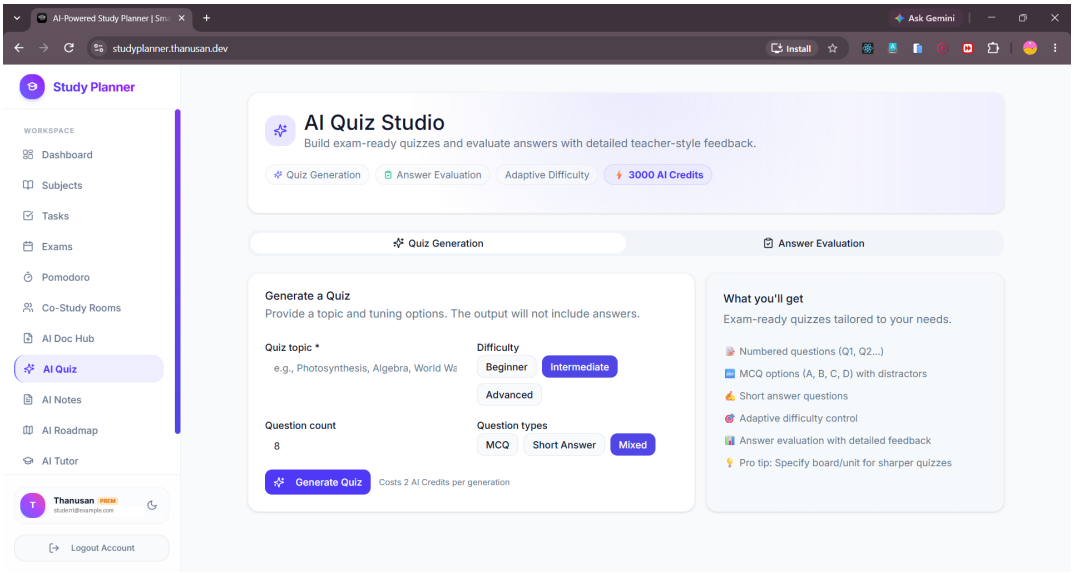


Figure 10: AI Quiz Studio: topic, difficulty, count and question types

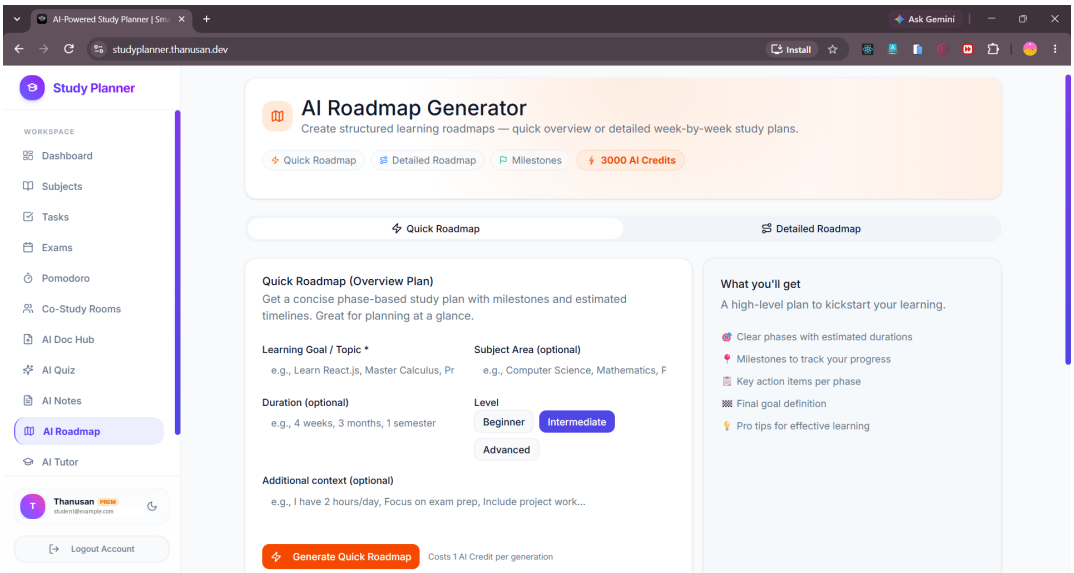


Figure 11: AI Roadmap Generator: quick and detailed modes

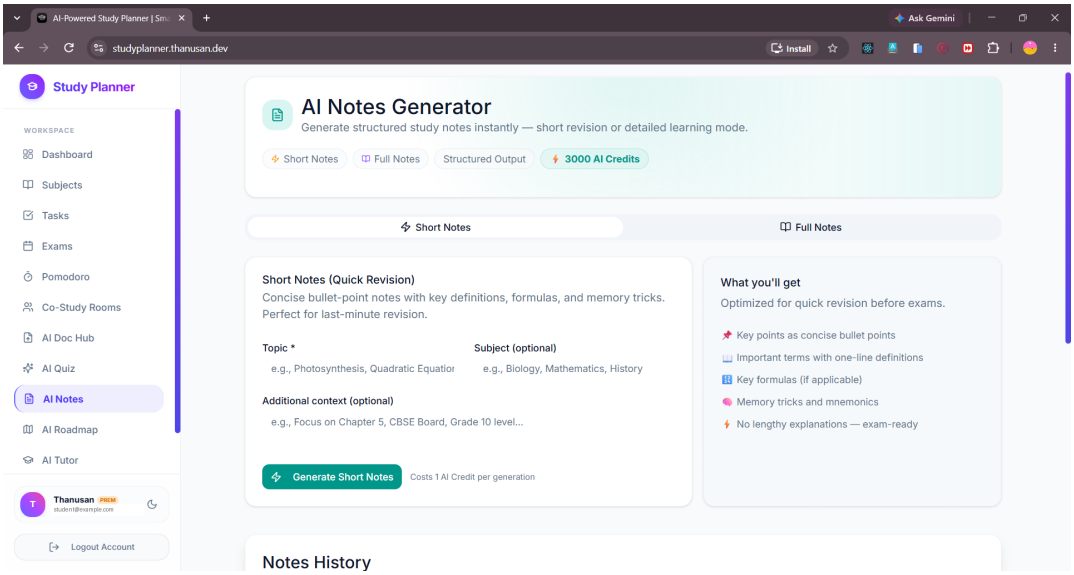


Figure 12: AI Notes Generator: short and full notes

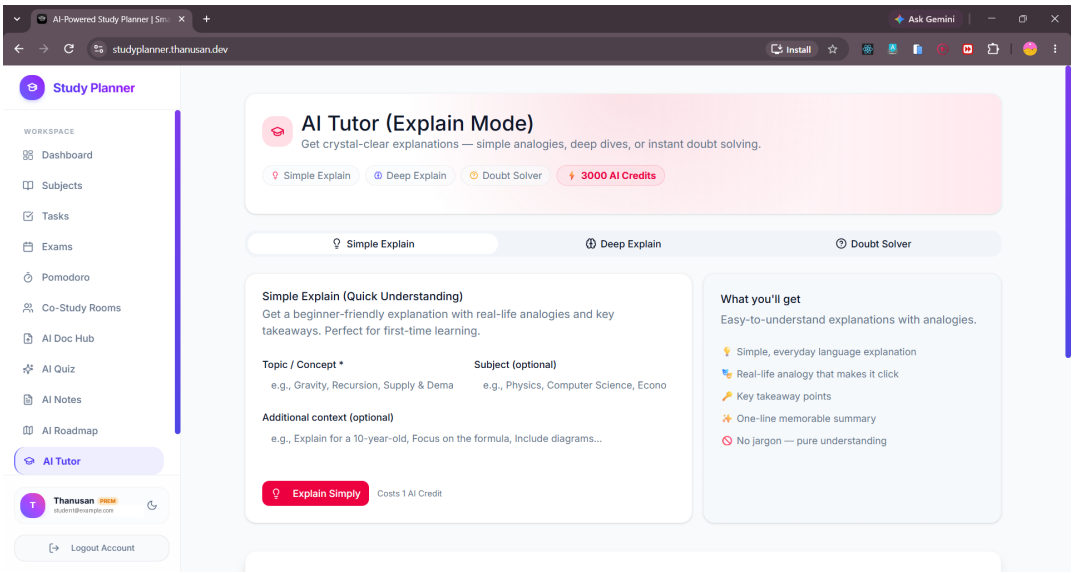


Figure 13: AI Tutor: simple explain, deep explain and doubt solver

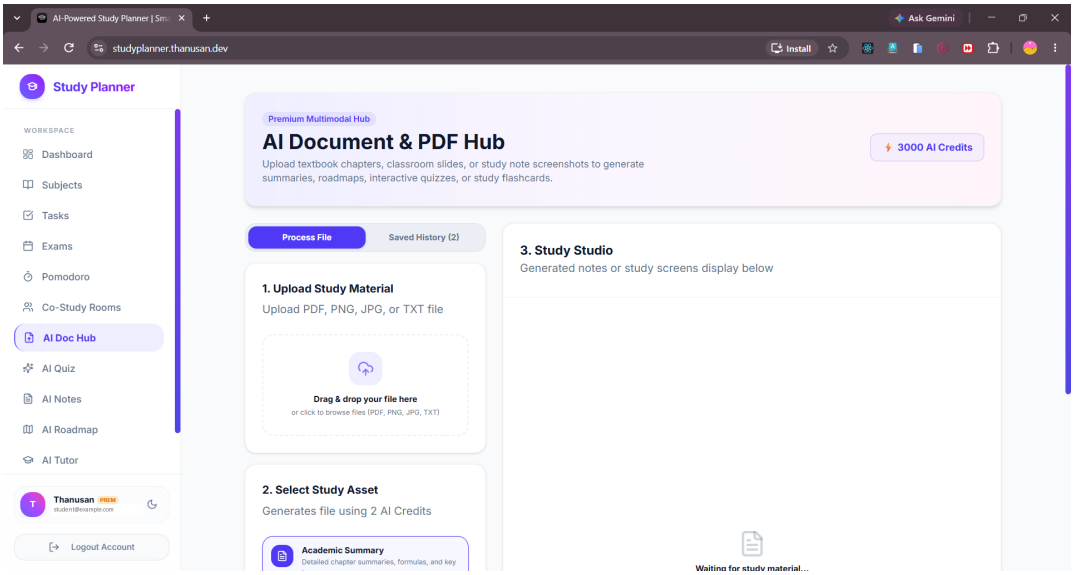


Figure 14: AI Document and PDF Hub

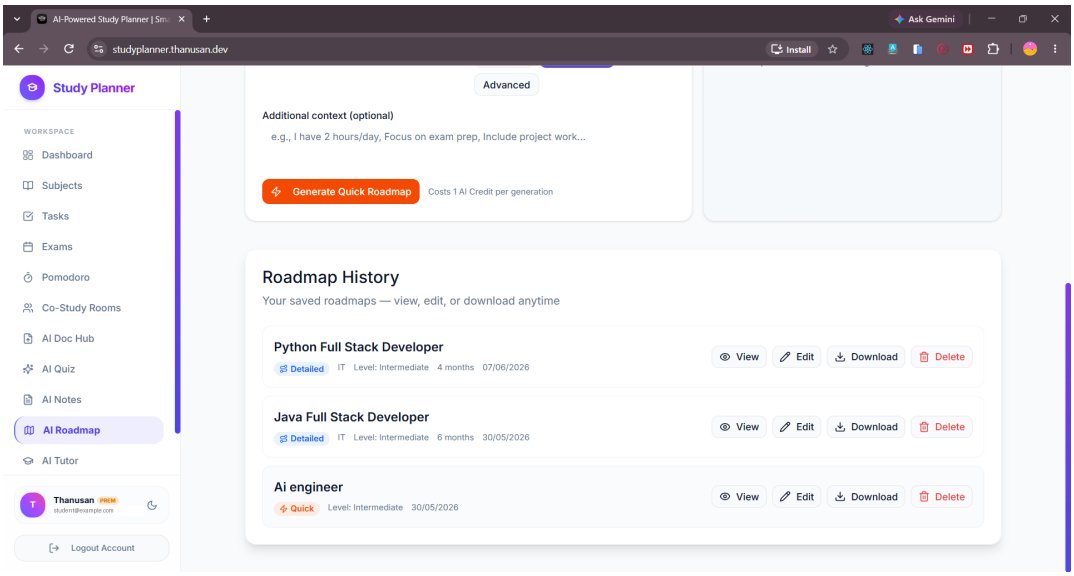


Figure 15: Roadmap history with view, edit, download and delete actions

7. Database Design and Integration

The application stores data in an Appwrite database. Appwrite organises records as collections of documents; the design follows relational principles, with a primary key (document ID) for each record and foreign-key attributes (*userId*, *subjectId*) that reference parent records. Figure 16 shows the entity-relationship model. A user owns many subjects, tasks, exams, sessions, quizzes, progress logs and ledger entries, and a subject owns many topics, tasks, exams and sessions.

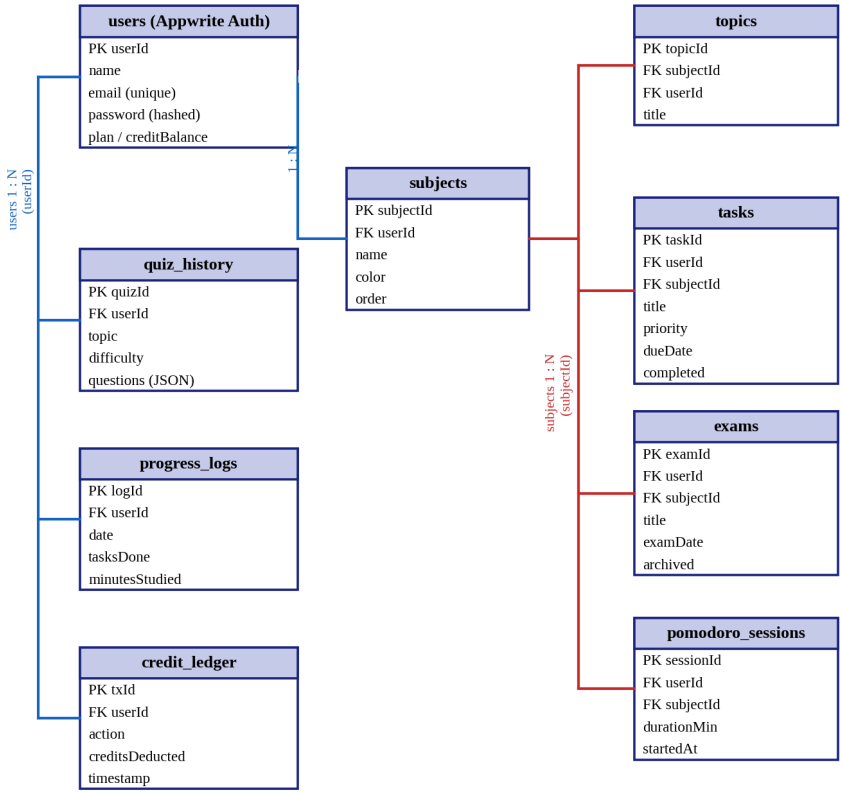


Figure 16: Entity-relationship diagram. Blue lines: relationship to users (userId); red lines: relationship to subjects (subjectId). See Appendix B for the SQL schema.

Integrity and performance. Required attributes, enumerated values (priority, difficulty) and string length limits are declared in the schema. Document-level permissions restrict each record to its owner. Indexes on *userId*, *dueDate* and *examDate* support fast filtered queries, and all list queries are paginated (limit 100). **CRUD evidence:** insertion (new task or subject), retrieval (lists and dashboards), update (completion toggle, edit, reschedule) and deletion (single and bulk) are shown in Figures 5, 6 and 7, and the Pomodoro and billing pages show stored and retrieved data after reloads. Seven primary collections are specified: Subjects, Topics, Tasks, Exams, ProgressLogs, PomodoroSessions and QuizHistory; the credit ledger, room and billing records support the premium features.

8. Authentication, Security, and Payment or API Integration

8.1 Authentication and authorization

Authentication uses Appwrite email/password accounts with session management and Google sign-in; passwords are salted and hashed by Appwrite and never stored by the application. Protected routes redirect unauthenticated visitors to the login page. Authorization is plan-based (Free, Scholar Pro, Elite Premium), so premium AI features are available only to eligible accounts, and data isolation ensures each user can read and write only their own documents.

8.2 Security measures

- Client-side and server-side validation of every form field.
- Secrets (Stripe keys, webhook secrets and service credentials) are held in environment variables inside functions and excluded from source control.
- HTTPS is enforced in production and Content Security Policy headers are recommended in the deployment guide.
- Notification access is permission-based and can be denied without breaking the application.
- Stripe payment events are processed by a server-side webhook function, not by the browser, before a plan or credit balance is changed.

8.3 Payment and API integration

Billing offers three plans: Free, Scholar Pro (LKR 1,500 per month) and Elite Premium (LKR 2,500 per month), with a yearly option that saves 33 percent. Checkout is created by the *stripe-checkout* function, and the *stripe-webhook* function processes payment status and updates the account. The billing page shows plan status, credit balance, the credit usage estimator, an auditable credit ledger and downloadable receipts (Figures 17 and 18). Payments are demonstrated in Stripe test mode, and Gemini API failures are reported to the user without charging credits.

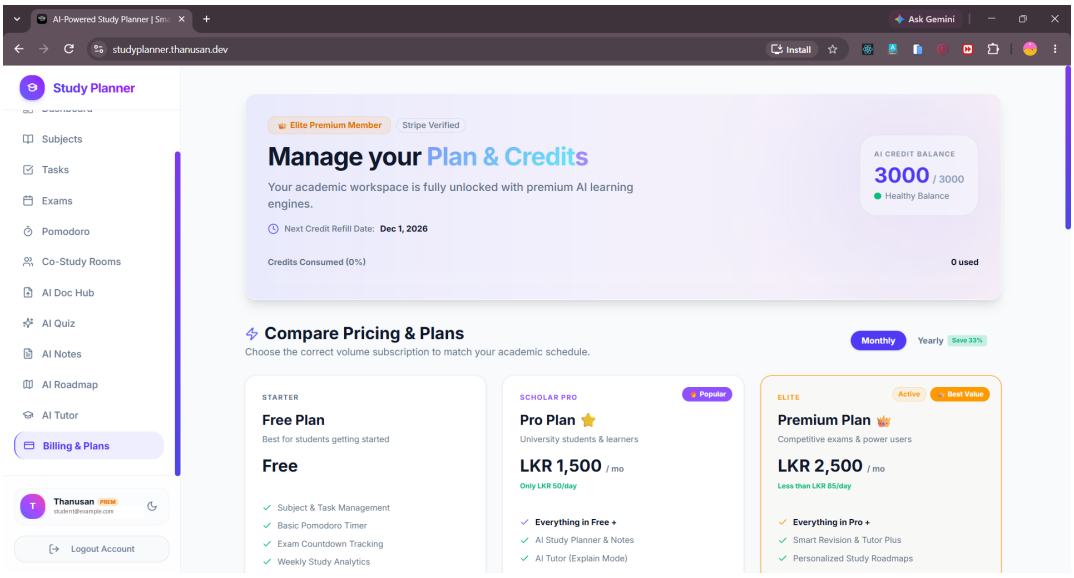


Figure 17: Billing and Plans page with credit balance and plan comparison

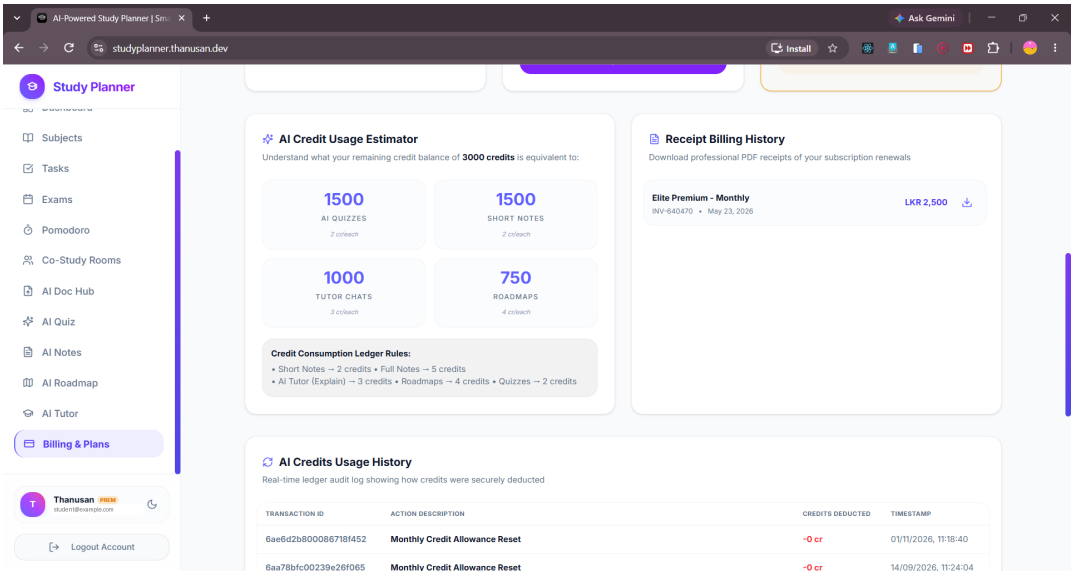


Figure 18: Credit usage estimator, receipt history and credit ledger

9. Testing and Quality Assurance

A test plan was prepared covering functionality, validation, database operations, authentication, authorization, API integration and error handling. Each case was executed on the deployed application using Google Chrome on Windows 11. the table below records the expected and actual results.

ID	Test case	Input / action	Expected result	Actual result	Status
T1	Valid login	Registered email and correct password	Dashboard opens with user name	Dashboard displayed (Figure 4)	Pass
T2	Invalid login	Correct email, wrong password	Error message; no access	Access refused with error message	Pass
T3	Empty required field	Submit AI Quiz with blank topic	Validation prevents submission	Request blocked; required field flagged	Pass
T4	Create task	Title, subject, priority, due date	Task saved and listed by due date	Task appears under correct date	Pass
T5	Complete task	Tick checkbox	Strikethrough and counts update	'1 of 1 completed'; dashboard 8/9 resolved	Pass
T6	Exam countdown	Exam dated 8 Nov 2026	Days remaining badge	'7 days' badge on 1 Nov 2026	Pass
T7	Delete subject	Delete icon with confirmation	Record removed	Subject removed after reload	Pass
T8	Pomodoro session	Start, finish 25-minute focus	Session stored; statistics update	Session history and totals updated	Pass
T9	AI generation	Valid quiz topic	Content returned; credits deducted	Content generated; ledger entry created	Pass

ID	Test case	Input / action	Expected result	Actual result	Status
T10	Plan authorization	Free account opens AI Tutor	Upgrade prompt shown	Premium feature blocked	Pass
T11	Study room chat	Send message in room	Message and join events shown to all	Messages and presence shown (Appendix A)	Pass
T12	Receipt download	Click receipt icon	PDF receipt downloads	Receipt file downloaded	Pass

Identified risks and corrective measures. Risk-driven checks were applied during development. Large result sets could slow list screens, so queries are paginated (limit 100) and indexed. Denied notification permission could break reminders, so notification calls are guarded by permission checks. Stale data after bulk updates is avoided by refreshing local state after each mutation, and failed AI requests do not deduct credits. These measures were verified in the test cases above.

10. Deployment and Production Readiness

The front end is built with *pnpm build* (output directory *dist*) and deployed to Vercel from the GitHub repository, so each push to the main branch produces a new build. The application is served over HTTPS at https://studyplanner.thanusan.dev. Appwrite Cloud hosts authentication, the database and functions; the project, database and collection identifiers are configured per environment, and production values may be supplied through Vite environment variables. Secret keys are set only in the function settings and are not committed to the repository.

Deployment steps are: (1) push the code to GitHub; (2) import the repository into Vercel and set the build command and output directory; (3) add environment variables; (4) create the Appwrite project, database, collections and indexes using the setup scripts; (5) deploy the four functions and configure their triggers; (6) register the Stripe webhook; (7) verify login, CRUD, AI generation, Pomodoro and billing on the live URL (Figures 2 to 18). **Limitations:** AI features depend on the Gemini API quota, payments run in Stripe test mode, and browser notifications require user consent.

11. GitHub Repository and Development Process

The code is maintained at https://github.com/ThanusanS/study-planner. The repository separates source (*src* with components, contexts, services, utilities and styles), serverless functions (*appwrite-functions*), setup scripts (*scripts*) and static assets (*public*), and includes *README.md*, *APPWRITE_SETUP.md*, *APPWRITE_FUNCTIONS.md*, *DEPLOYMENT.md* and *QUICKSTART.md*. The README documents features, technology stack, database collections, configuration, deployment, security, performance targets and troubleshooting. Development followed the weekly evaluation schedule: planning, front end, back end and database, core features and security, then testing and documentation, with commits made after each increment. The application was built with Gemini 2.5 Flash as its AI model and the work is my own; all external libraries and services are acknowledged in the references.

12. Conclusion

The project delivers a complete, deployed full-stack application that meets the assessment objectives: a responsive interface, documented architecture, Appwrite-backed persistence with CRUD operations, authentication and plan-based authorization, Gemini AI integration, Stripe payments in test mode, structured testing and continuous deployment from GitHub. The principal lesson was that managed back-end services and a clear service layer let a single developer focus on user value. Future work includes offline mode, calendar integration, gamification, data export and a native mobile client.

13. References

Appwrite (2026) *Appwrite documentation*. Available at: https://appwrite.io/docs (Accessed: 1 November 2026).

Cirillo, F. (2018) *The Pomodoro Technique*. London: Penguin Random House.

Google (2026) *Gemini API documentation*. Available at: https://ai.google.dev/gemini-api/docs (Accessed: 1 November 2026).

Meta (2026) *React documentation*. Available at: https://react.dev (Accessed: 1 November 2026).

shadcn (2026) *shadcn/ui components*. Available at: https://ui.shadcn.com (Accessed: 1 November 2026).

Sommerville, I. (2016) *Software engineering*. 10th edn. Harlow: Pearson.

Stripe (2026) *Stripe documentation: Checkout and webhooks*. Available at: https://docs.stripe.com (Accessed: 1 November 2026).

Tailwind Labs (2026) *Tailwind CSS documentation*. Available at: https://tailwindcss.com/docs (Accessed: 1 November 2026).

Vercel (2026) *Vercel documentation*. Available at: https://vercel.com/docs (Accessed: 1 November 2026).

14. Appendices

Appendix A: Additional application screenshots

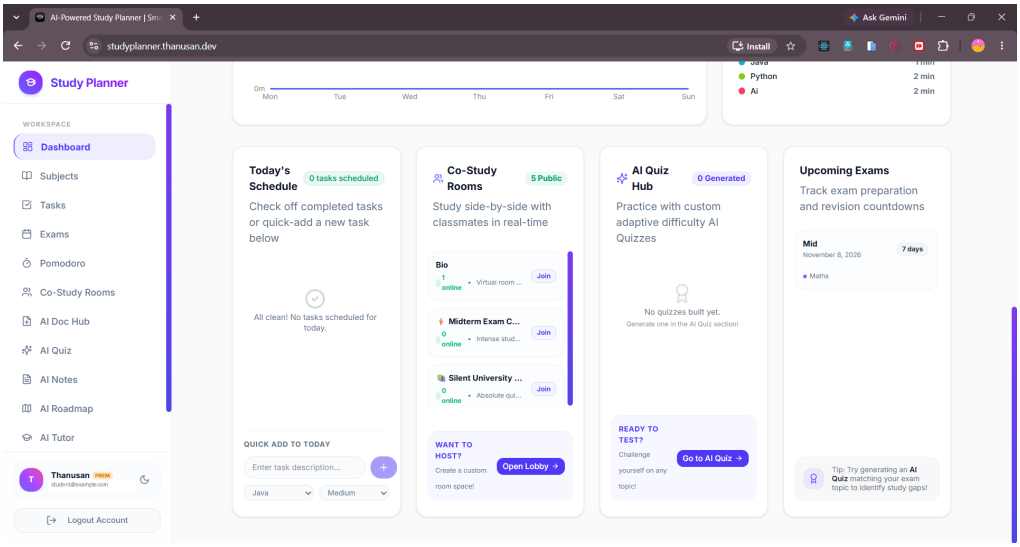


Figure 19: Dashboard: today's schedule, co-study rooms, AI quiz hub and upcoming exams

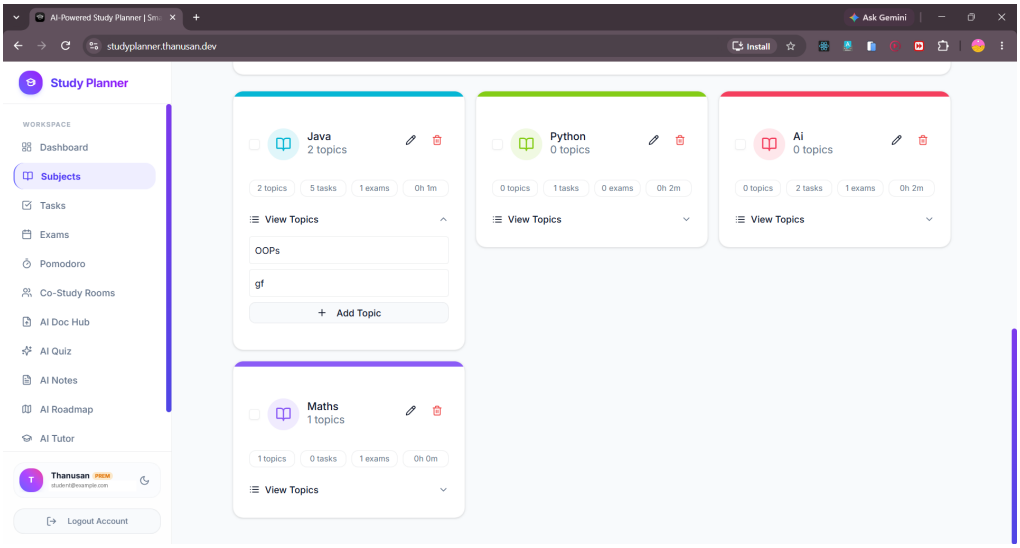


Figure 20: Subject cards with topics and Add Topic control

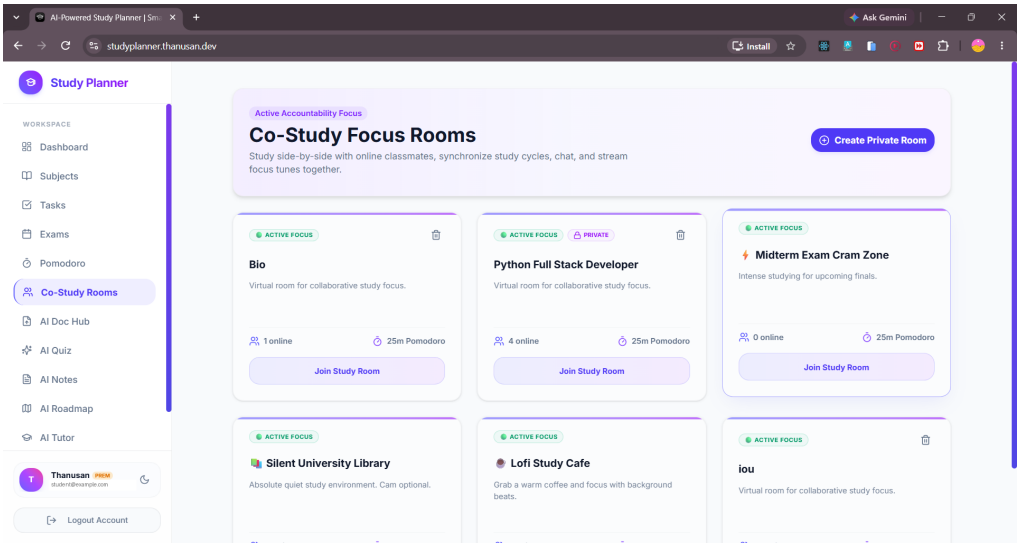


Figure 21: Co-Study Focus Rooms lobby with public and private rooms

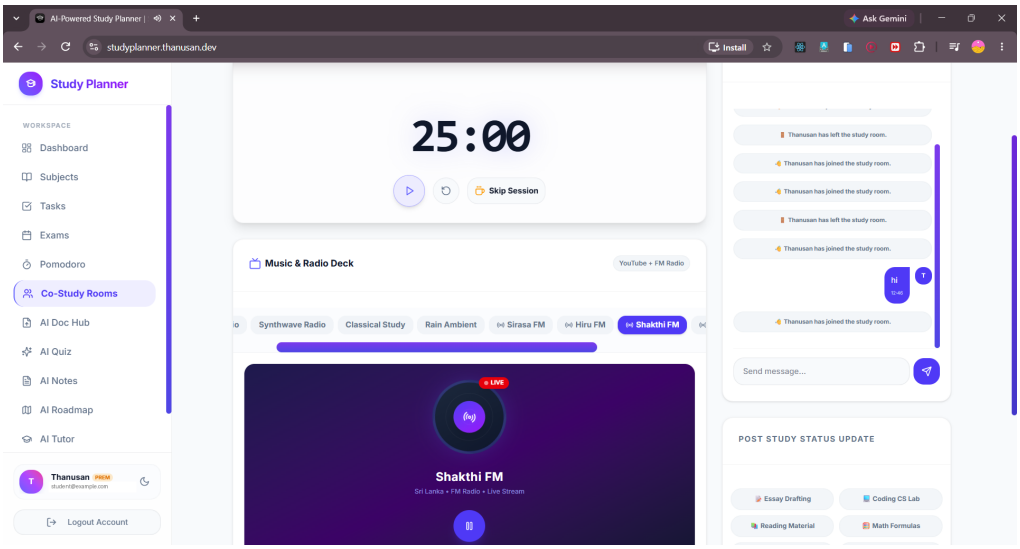


Figure 22: Inside a study room: synchronised timer, music and radio deck, chat and presence events

Appendix B: Equivalent relational (SQL) schema

Appwrite stores documents rather than SQL tables. The statements below express the same entities, keys and constraints in SQL (PostgreSQL syntax) for documentation and for demonstrating relational design. The live system implements them as Appwrite collections, attributes, indexes and permissions.

```
CREATE TABLE users (
  user_id       VARCHAR(36) PRIMARY KEY,
  name          VARCHAR(100) NOT NULL,
  email         VARCHAR(255) UNIQUE NOT NULL,
  password_hash VARCHAR(255) NOT NULL,
  plan          VARCHAR(20) NOT NULL DEFAULT 'free',
  credit_balance INT NOT NULL DEFAULT 0 CHECK (credit_balance >= 0)
);
CREATE TABLE subjects (
  subject_id VARCHAR(36) PRIMARY KEY,
  user_id    VARCHAR(36) NOT NULL REFERENCES users(user_id) ON DELETE CASCADE,
  name       VARCHAR(100) NOT NULL,
  color      VARCHAR(9),
  sort_order INT DEFAULT 0
```

```
);
CREATE TABLE topics (
  topic_id   VARCHAR(36) PRIMARY KEY,
  subject_id VARCHAR(36) NOT NULL REFERENCES subjects(subject_id) ON DELETE CASCADE,
  user_id    VARCHAR(36) NOT NULL REFERENCES users(user_id) ON DELETE CASCADE,
  title      VARCHAR(150) NOT NULL
);
CREATE TABLE tasks (
  task_id    VARCHAR(36) PRIMARY KEY,
  user_id    VARCHAR(36) NOT NULL REFERENCES users(user_id) ON DELETE CASCADE,
  subject_id VARCHAR(36) REFERENCES subjects(subject_id) ON DELETE SET NULL,
  title      VARCHAR(200) NOT NULL,
  priority   VARCHAR(10) CHECK (priority IN ('low','medium','high')),
  due_date   DATE,
  completed  BOOLEAN NOT NULL DEFAULT FALSE
);
CREATE TABLE exams (
  exam_id    VARCHAR(36) PRIMARY KEY,
  user_id    VARCHAR(36) NOT NULL REFERENCES users(user_id) ON DELETE CASCADE,
  subject_id VARCHAR(36) REFERENCES subjects(subject_id) ON DELETE SET NULL,
  title      VARCHAR(150) NOT NULL,
  exam_date  DATE NOT NULL,
  archived   BOOLEAN NOT NULL DEFAULT FALSE
);
CREATE TABLE pomodoro_sessions (
  session_id   VARCHAR(36) PRIMARY KEY,
  user_id      VARCHAR(36) NOT NULL REFERENCES users(user_id) ON DELETE CASCADE,
  subject_id   VARCHAR(36) REFERENCES subjects(subject_id) ON DELETE SET NULL,
  duration_min INT NOT NULL CHECK (duration_min > 0),
  started_at   TIMESTAMP NOT NULL
);
CREATE TABLE quiz_history (
  quiz_id    VARCHAR(36) PRIMARY KEY,
  user_id    VARCHAR(36) NOT NULL REFERENCES users(user_id) ON DELETE CASCADE,
  topic      VARCHAR(200) NOT NULL,
  difficulty VARCHAR(15),
  questions  JSON NOT NULL
);
CREATE TABLE progress_logs (
  log_id          VARCHAR(36) PRIMARY KEY,
  user_id         VARCHAR(36) NOT NULL REFERENCES users(user_id) ON DELETE CASCADE,
  log_date        DATE NOT NULL,
  tasks_done      INT DEFAULT 0,
  minutes_studied INT DEFAULT 0
);
CREATE TABLE credit_ledger (
  tx_id           VARCHAR(36) PRIMARY KEY,
  user_id         VARCHAR(36) NOT NULL REFERENCES users(user_id) ON DELETE CASCADE,
  action          VARCHAR(150) NOT NULL,
  credits_deducted INT NOT NULL,
  created_at      TIMESTAMP NOT NULL DEFAULT NOW()
);
CREATE INDEX idx_tasks_user_due ON tasks(user_id, due_date);
CREATE INDEX idx_exams_user_date ON exams(user_id, exam_date);
```

Appendix C: Example CRUD statements

```
-- Create
INSERT INTO tasks (task_id, user_id, subject_id, title, priority, due_date)
VALUES ('t001','u001','s001','Java intro','medium','2026-05-05');
-- Read (tasks grouped by due date for one user)
SELECT due_date, title, completed FROM tasks
WHERE user_id = 'u001' ORDER BY due_date;
-- Update (one-click completion)
UPDATE tasks SET completed = TRUE WHERE task_id = 't001' AND user_id = 'u001';
-- Delete
DELETE FROM tasks WHERE task_id = 't001' AND user_id = 'u001';
```

Appendix D: Application configuration (no secrets)

```
export const appwriteConfig = {
  endpoint:   import.meta.env.VITE_APPWRITE_ENDPOINT || "https://cloud.appwrite.io/v1",
  projectId:  import.meta.env.VITE_APPWRITE_PROJECT_ID,
  databaseId: import.meta.env.VITE_APPWRITE_DATABASE_ID,
  // collection IDs ...
};
```

Appendix E: Serverless functions

Function	Purpose
stripe-checkout	Creates a Stripe checkout session for the selected plan.
stripe-webhook	Verifies Stripe events and updates plan and credit allowance.
task-reminders	Scheduled function that prepares task reminder notifications.
exam-alerts	Scheduled function that raises alerts for upcoming exams.